Makefile 笔记

一、 Makefile 的核心逻辑：规则 (Rule)

Makefile 的本质是由一条条**规则**组成的。

1. 规则格式

一条规则的结构如下：

```
目标 (target) ... : 依赖 (prerequisites) ...
    命令 (command)
    ...
```

- **目标 (Target)：** 通常是要生成的文件名（如可执行文件、`.o` 文件），也可以是一个动作名称（如 `clean`）。
- **依赖 (Prerequisites)：** 生成目标所需要的文件或其它目标。
- **命令 (Command)：** 只要依赖文件比目标文件更新，或者目标文件不存在，Make 就会执行这些 Shell 命令。

2. 致命细节：Tab 缩进

在 Makefile 中，**命令前面必须是一个 Tab 键，绝对不能是空格。**如果误用了空格，Make 会报错 `makefile: *** missing separator.  Stop.`。

3. 注释与换行

- `#` 用来写注释，整行或行尾都可以。
- 如果一条命令太长，可以用反斜杠 `\` 续行。

```
all:
    gcc -wall -g \
        -I./include \
        -o app main.c util.c
```

二、 命令前的 `@` 符号与 `@echo`

当你运行 `make` 时，默认会把执行的命令本身打印到终端。

- `echo "Hello"`：终端会显示 `echo "Hello"` 然后再显示 `Hello`。
- `@echo "Hello"`：在命令前加上 `@`，表示**静默执行。**终端只会显示 `Hello`，不显示命令本身。这常用于美化编译输出。

三、 Makefile 变量

1. 变量定义与使用

变量就像是 C 语言中的宏，可以简化代码。**引用变量必须使用** `$(变量名)`。

```
CC = gcc              # 定义变量
TARGET = main

$(TARGET): main.c     # 使用变量
    $(CC) -o $(TARGET) main.c
```

2. 赋值符号的区别

- `=`：**延时赋值**。变量的值取整个 Makefile 最后决定的值。
- `:=`：**立即赋值**。变量的值在定义处立即决定。
- `?=`：**条件赋值**。如果变量没被定义过，才给它赋值。
- `+=`：**追加赋值**。在原值后面加上新内容。

```
# 1) `=` 延时赋值：后面改了 BAR，FOO 也会跟着变
FOO = $(BAR)
BAR = hello
BAR = world

# 2) `:=` 立即赋值：定义时就把当前值固定下来
BAZ := $(BAR)
BAR = changed

# 3) `?=` 条件赋值：只有没定义过时才会生效
DEBUG ?= 0
DEBUG ?= 1

# 4) `+=` 追加赋值：在原有内容后继续加选项
CFLAGS += -Wall
CFLAGS += -g
```

```
all:
  @echo "FOO=$(FOO)"
  @echo "BAZ=$(BAZ)"
  @echo "DEBUG=$(DEBUG)"
  @echo "CFLAGS=$(CFLAGS)"
```

3. 常见内置变量

除了自己定义的变量，Makefile 里还有一些很常用的内置变量：

- `CC`：编译器，通常是 `gcc` 或交叉编译器。
- `CFLAGS`：C 编译选项，常放 `-Wall -g -O2` 之类参数。
- `CPPFLAGS`：预处理选项，常放 `-I` 和 `-D`。
- `LDFLAGS`：链接选项，常放库路径 `-L`。

- `LDLIBS` 或 `LIBS`：链接库，常放 `-lm`、`-lpthread` 等。

4. 变量展开的两个常见写法

- `=`：递归展开，变量在使用时才确定最终值。
- `:=`：简单展开，变量在定义时就确定最终值。

```
FOO = $(BAR)
BAR = hello

BAZ := $(BAR)
BAR = world
```

这里 `FOO` 最终会变成 `hello`，`BAZ` 仍然是 `hello`。

四、 自动化变量：四两拨千斤

自动化变量是 Makefile 的“特种兵”，它们能根据规则自动替换内容。

变量	含义
`$@`	表示规则中的**目标文件**
`$%`	表示归档成员文件名，常用于静态库
`$<`	表示规则中的**第一个依赖文件**
`$^`	表示规则中**所有的依赖文件**，以空格分隔
`$?`	表示**比目标文件更新的依赖文件**
`$+`	表示**所有依赖文件**，会保留重复项
`$*`	表示**目标文件的“主干名”**，常见于模式规则
`$(@D)`	目标文件所在目录
`$(@F)`	目标文件文件名部分
`$(<D)`	第一个依赖所在目录
`$(<F)`	第一个依赖文件名部分

举例说明如何运用

假设我们要编译 `main.o`，依赖是 `main.c` 和 `header.h`：

```
# 原始写法
main.o: main.c header.h
    gcc -c main.c -o main.o

# 使用自动化变量优化后
main.o: main.c header.h
    gcc -c $< -o $@   # $< 是 main.c, $@ 是 main.o
```

再补两个常见场景

```
# 只重新编译有更新的依赖
main.o: main.c header.h
 @echo "Changed files: $?"

# 链接时保留重复库顺序
app: main.o util.o
 gcc -o $@ $+
```

- `$?` 常用于增量编译，只处理比目标新的文件。
- `$+` 常用于链接阶段，尤其是依赖顺序不能乱的时候。
- `$(@D)`、`$(@F)` 适合把生成物放进单独目录里。

```
obj/main.o: src/main.c include/header.h
 @mkdir -p $(@D)
 $(CC) -c $< -o $@
```

五、 模式规则 (Pattern Rules)

如果你有 100 个 `.c` 文件，不需要写 100 遍规则。使用 `%` 通配符：

```
# 所有的 .o 文件都依赖于对应的 .c 文件
%.o: %.c
 $(CC) -c $< -o $@
```

这里的 `%` 会匹配相同的文件名。

也可以给一组明确目标写静态模式规则：

```
OBJS = main.o util.o

$(OBJS): %.o: %.c
 $(CC) -c $< -o $@
```

六、 伪目标 (.PHONY)

如果目录下恰好有一个叫 `clean` 的文件，你执行 `make clean` 时，Make 会认为 `clean` 文件已存在且没有依赖，所以“不更新”，导致命令不执行。
为了避免这种尴尬，我们需要声明**伪目标**：

```
.PHONY: clean

clean:
 rm -f *.o $(TARGET)
```

声明后，即使目录下有 `clean` 文件，`make clean` 也会照常运行。

常见伪目标还有 `all`、`install`、`test`、`distclean`。

七、 条件判断

类似于 C 语言的预编译处理，常见写法有 `ifeq`、`ifneq`、`ifdef` 和 `ifndef`：

```
ARCH ?= x86

ifeq ($(ARCH), arm)
    CC = arm-linux-gnueabihf-gcc
else
    CC = gcc
endif

ifdef DEBUG
 CFLAGS += -g -O0
else
 CFLAGS += -O2
endif
```

八、 常用函数

Makefile 提供了很多内置函数，下面这些最常见：

- `wildcard`：获取匹配模式的所有文件名。
  - `SRC = $(wildcard *.c)` -> 获取当前目录下所有 `.c` 文件。
- `patsubst`：模式替换。
  - `OBJ = $(patsubst %.c, %.o, $(SRC))` -> 把 `SRC` 中所有的 `.c` 替换成 `.o`。
- `filter`：保留符合模式的字符串。
  - `CFILES = $(filter %.c, $(SRC))`
- `filter-out`：去掉符合模式的字符串。
  - `NO_TEST = $(filter-out test%.c, $(SRC))`
- `addprefix`：给每个单词加前缀。
  - `OBJS = $(addprefix obj/, $(OBJ))`
- `addsuffix`：给每个单词加后缀。
  - `FILES = $(addsuffix .bak, $(SRC))`
- `dir`：提取目录名。
  - `$(dir src/main.c)` -> `src/`

- `notdir`：去掉目录部分。
  - `$(notdir src/main.c)` -> `main.c`
- `basename`：去掉后缀。
  - `$(basename main.c)` -> `main`
- `subst`：字符串替换。
  - `$(subst .c,.o,main.c)` -> `main.o`
- `shell`：执行 Shell 命令并返回结果。
  - `$(shell pwd)` -> 当前目录
- `foreach`：遍历列表。
  - `$(foreach f,$(SRC),$(f))`

九、 特殊目标与默认目标

- `.DEFAULT_GOAL`：指定默认执行目标。
- `.DELETE_ON_ERROR`：目标生成失败时删除已生成文件，避免留下坏文件。
- `.SECONDARY`：保留中间文件，方便调试。

```
.DEFAULT_GOAL := all
.DELETE_ON_ERROR:
```

十、 include 与路径查找

Makefile 可以把公共变量拆到单独文件里，也可以指定默认搜索路径：

- `include`：包含其他 Makefile 文件。
- `-include`：即使文件不存在也不报错。
- `VPATH`：告诉 Make 去哪些目录找依赖文件。
- `vpath`：更细粒度地为某类文件指定搜索路径。

```
include config.mk
-include local.mk

VPATH := src:include
```

十一、 综合示例（嵌入式开发常用模板）

```
# 定义变量
CC = arm-linux-gnueabihf-gcc
TARGET = app
BUILD_DIR = build
CPPFLAGS += -I./include
CFLAGS += -Wall -g
LDFLAGS += -L./lib
```

```makefile
LDLIBS += -lm
SRCS = $(wildcard *.c)
OBJS = $(patsubst %.c, %.o, $(SRCS))

.DEFAULT_GOAL := $(TARGET)

# 默认目标
$(TARGET): $(OBJS)
 $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)
 @echo "Build successful: $@"

# 模式规则
%.o: %.c
 $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

# 伪目标
.PHONY: clean
clean:
 rm -f $(OBJS) $(TARGET)

.PHONY: all
all: $(TARGET)
```